Manipulation des données & Gestion des transactions

Définition détaillée de “Transaction” en MySQL

Définition :

Une **transaction** est un ensemble d’opérations SQL qui doivent être exécutées ensemble, comme un seul bloc indivisible.

➞ Soit tout est exécuté avec succès → **COMMIT**
➞ Soit une seule opération échoue → **ROLLBACK** annule tout

Une **transaction** MySQL est un mécanisme de sécurité qui garantit que les données de la base restent toujours correctes, cohérentes et fiables, même en cas de :

- **panne d’électricité**
- **crash du serveur**
- **erreur du programme**
- **problème réseau**
- **interruption du client**
- **erreur dans une requête**

Une **transaction** :

✓ **Regroupe plusieurs opérations SQL en une seule ACTION logique**

Ex :
Débiter → Créditer → Inscrire la trace du paiement

Ces trois actions = une seule transaction.

✓ Empêche votre base d'être dans un état incomplet ou incohérent

Sans transaction :

- Débit réussi
- Crédit échoué
  tu perds 200 DH !

Avec transaction :
➞ le **ROLLBACK** annule automatiquement le débit

BEGIN;

*On commence une **transaction**.*
MySQL dit :

"OK, j'attends toutes les opérations. Je ne valide rien pour l'instant."

1) Débit : réussi

Le montant est retiré du compte A, **mais pas encore confirmé**.

2) Crédit : échoué

Une erreur arrive.

ROLLBACK;

On annule TOUT ce qui s'est passé depuis le BEGIN.

✓ Repose sur 4 propriétés fondamentales (ACID)

Lettre	Propriété	Explication
A	**Atomicité**	La transaction est indivisible : tout ou rien
C	**Cohérence**	La base doit rester correcte avant et après
I	**Isolation**	Les transactions ne se voient pas entre elles avant le commit
D	**Durabilité**	Une fois commit, les données sont gravées définitivement

✓ Contrôle l'autocommit

MySQL valide automatiquement chaque commande (INSERT/DELETE/UPDATE).
Pour activer les transactions réelles :

SET autocommit = 0;

Les commandes TRANSACTION

1. START TRANSACTION

Je commence une transaction, c'est moi qui décide quand ça valide.

2. COMMIT

Je valide définitivement toutes les opérations effectuées depuis le START TRANSACTION.

3. ROLLBACK

J'annule toutes les opérations effectuées depuis START TRANSACTION, comme si rien ne s'était passé.

Exemple pratique (banque)

Requête :

```
START TRANSACTION;
UPDATE compte
SET solde = solde - 200
WHERE accountno = 'ACC1';
UPDATE compte
SET solde = solde + 200
WHERE accountno = 'ACC2';
COMMIT;
```

➞ Démarre une transaction : rien ne sera enregistré tant que COMMIT n'est pas exécuté.

➞ Débite 200 DH du compte ACC1 (retrait provisoire, pas encore validé).

➞ Crédit 200 DH au compte ACC2 (ajout provisoire en attente de validation).

➞ Valide toutes les opérations : le retrait et le dépôt deviennent définitifs

Version avec ROLLBACK :

START TRANSACTION;

UPDATE compte SET solde = solde - 200 WHERE accountno = 'ACC1';

UPDATE compte SET solde = solde + 200 WHERE accountno = 'ACC2';

ROLLBACK;

➞ La transaction est annulée.

➞ Le solde de ACC1 redevient comme avant.

➞ Le crédit prévu pour ACC2 est supprimé.

➞ La base revient exactement à l'état initial, comme si rien n'avait été exécuté.

➞ C'est la grande force des transactions.

Processus interne (ce que MySQL fait en coulisses)

1. **START TRANSACTION**
   - MySQL crée un espace mémoire pour stocker les modifications.
   - Aucune écriture réelle dans les fichiers de données.
2. **Exécution des UPDATE**
   - MySQL applique les modifications en mémoire.
   - Les autres sessions ne voient pas encore les modifications (isolation).
3. **COMMIT**

- MySQL enregistre les données dans les fichiers réels.
- Les verrous sont libérés.
- Les autres sessions voient les nouvelles valeurs.

4. ROLLBACK

- MySQL supprime simplement les modifications temporaires.
- Aucune écriture réelle dans la base.
- Retour exact à l'état initial.

Autocommit

Par défaut : **AUTOCOMMIT = ON**

Chaque commande SQL est validée automatiquement.

INSERT → validé automatiquement

UPDATE → validé automatiquement

DELETE → validé automatiquement

- **Nb : Impossible de revenir en arrière.**

Désactiver l'autocommit

SET **autocommit = 0;**

➞ Toutes les opérations doivent être terminées par **COMMIT.**

Exemple avec INSERT / DELETE / UPDATE

```
START TRANSACTION;                    -- Début de la transaction
INSERT INTO clients(id, nom)
VALUES (101, 'Said');                 -- Ajout d'un client

UPDATE clients
SET nom = 'Said Slaoui'
WHERE id = 101;                       -- Modification du même client

DELETE FROM commandes
WHERE client_id = 101;                -- Suppression éventuelle des commandes liées

COMMIT;                               -- Validation finale de toutes les opérations
```

Erreurs fréquentes à éviter :

Oublier WHERE dans UPDATE ou DELETE

→ Sans transaction, c'est irréversible

→ Avec ROLLBACK = sauvé

Faire COMMIT trop tôt

→ Impossible de revenir en arrière

Utiliser **COMMIT** après une seule commande

→ **Cela perd l'intérêt des transactions**

<u>En résumé :</u>

- **Une transaction** = ensemble d'opérations qui doivent toutes réussir ensemble
- **START TRANSACTION** = début
- **COMMIT** = valider définitivement
- **ROLLBACK** = annuler totalement
- **autocommit** = ON empêche d'utiliser des transactions correctement
- **Cas d'utilisation fréquent** : transferts bancaires, paiements, commandes

OFPPT

Formateur : AARAB HAFID